

Geo-Engine Terrain Classifier - Performance Roadmap

Fast, memory-safe 10 m terrain classification at district scale
(research report)

2026-08-14

Executive summary

1. How the system works today

- 1.1 Backend pipeline
- 1.2 Frontend (v1, Leaflet)
- 1.3 What was already tried

2. Diagnosis: where the 40 minutes and the memory go

3. Design goals and constraints

4. Recommended target architecture

5. Compute: killing the 40-minute wait

- 5.1 Option A (stopgap, ~1-2 days): one Earth Engine batch export to COG
- 5.2 Option B (recommended target, ~1.5-3 weeks): move feature building and inference off Earth Engine
- 5.3 Option C (multiplies B): cached per-district feature cubes
- 5.4 Performance table (district-size AOI)
- 5.5 Compute references

6. Storage: the Cloud Optimized GeoTIFF

- 6.1 Storage COG references

7. Serving: TiTiler behind nginx + CDN

- 7.1 Why TiTiler
- 7.2 Process shape and GDAL tuning
- 7.3 Cache hierarchy
- 7.4 Serving references

8. Orchestration: long jobs that survive restarts

- 8.1 Orchestration references

9. Client: MapLibre GL raster tiles

- 9.1 Recommended stack
- 9.2 Source and layer settings
- 9.3 Memory budget
- 9.4 Alternative clients ranked
- 9.5 Client references

10. Consolidated expected performance

11. Migration roadmap

- 12. Risks and open questions

13. Technology stack summary and research basis

Executive summary

Rendering a district the size of Jabalpur (~105 megapixels at 10 m) as a classified overlay currently takes **~40 minutes** and then freezes the browser with **~420 MB of decoded image** in memory. The two problems are separate and both fixable with current, mature geospatial technology:

1. **The compute bottleneck.** The backend re-runs the entire Earth Engine inference chain (Sentinel-2 composite, spectral indices, GLCM texture, Sentinel-1 SAR, classifier) for every one of ~50 tiled requests, is rate-limited to 3 concurrent calls by Earth Engine's Restricted Mode, and stitches the output into one giant PNG.
2. **The delivery and rendering bottleneck.** That giant PNG (~100 MP) is shipped whole to the browser, which decodes it as one ~420 MB RGBA image and re-rasterises it on every pan and zoom.

The recommended direction, in one line:

Compute the classification once per AOI at 10 m, store it as a Cloud Optimized GeoTIFF (COG), serve it as small XYZ map tiles, and render those tiles with MapLibre GL.

With the fastest compute option (inference moved off Earth Engine into your own Python process) the expected change is:

Metric	Today	Target
Jabalpur district end-to-end wall clock	~40 min	~2-6 min cold, cached afterwards
Re-run of a cached AOI (same params)	paid again	instant (hash-keyed cache)
Time-to-first-map after job finishes	waits for the whole PNG	seconds; tiles stream in
Browser tab memory	~420 MB decoded giant PNG	~150-300 MB total; ~15-25 MB GPU for the overlay
Pan / zoom smoothness	janky, full-image re-raster	60 fps, only visible tiles
Earth Engine quota burn per render	high, months in Restricted Mode	~0 on the off-GEE path
Tile latency	whole-image, n/a	~20-70 ms cold, milliseconds cached

This report is the synthesis of four parallel research threads: Cloud Optimized GeoTIFF serving, client-side rendering, compute-side acceleration, and job orchestration/infrastructure. It is a research and architecture document, not an implementation. Claims that could not be confirmed against a primary source are flagged. Each section ends with references.

1. How the system works today

1.1 Backend pipeline

The request path for “classify this district” runs entirely inside one FastAPI process on Google Earth Engine (GEE), in `backend/gee_classifier.py` :

1. **Train (cached).** `get_trained_classifier` merges the labelled training points, builds a Sentinel-2 composite over the training bounding box, then trains one of five GEE classifiers (Random Forest default, 200 trees). The result is cached in-process keyed by `(model, dates, cloud, schema)`, so it trains once per process, not per request. First training of a cold model+date window can take up to ~140 s.
2. **Build the AOI composite.** `build_sentinel_composite` filters Sentinel-2 (`COPERNICUS/S2_SR_HARMONIZED`), median-composites, and adds ~17 derived bands: NDVI/NDWI/NDBI/SAVI, BSI/UI/IBI/SWIRratio/BAEI, GLCM texture over a 3x3 pixel window on B8 (32 grey levels), and Sentinel-1 SAR (VV/VH + ratio + texture). Total 19-band feature stack.
3. **Classify at 10 m.** `_classify` runs the classifier plus optional `focalMode` square-1px smoothing. The output is **only valid at the 10 m training scale**: GLCM and focal filters are defined on a *pixel* grid, so asking for the classification at any other resolution silently re-runs the chain there and produces a *different* classification (the code documents water share varying 5.7% at 10 m to 32% at 100 m). This is the hard constraint the legacy rendering approach was built around.
4. **Render as tiles.** GEE refuses one ~105 MP synchronous request (“Reprojection output too large”). `render_overlay_png` cuts the AOI into 1792x1792 px tiles on the Web-Mercator grid and calls `getThumbURL` per tile at 10 m. Measured behavior: ~2 min per tile, HTTP 429 above 3 concurrent (`RENDER_WORKERS=3`). A Jabalpur-sized district is ~50 tiles, so the render takes **~25-40 minutes** on a daemon thread while the frontend polls.
5. **Stitch and serve.** Pillow stitches the tiles into one RGBA PNG (a 9,600 x 5,400 px district is ~420 MB in the canvas) and caches it to `.overlay_cache/{key}.png`, served flat through a static mount. The class histogram (`reduceRegion`) and GeoTIFF export URL are computed synchronously in the same request.

1.2 Frontend (v1, Leaflet)

The user draws an AOI and hits classify. The map shows a single `L.imageOverlay` of the stitched PNG; a progress banner polls `/api/overlay/{key}` every 5 s until the backend is done. Panning and zooming re-rasterise the one huge `<img>` every frame, and the browser keeps the whole decoded raster resident, which is the reported lag and memory blowout.

1.3 What was already tried

`doc/frontend_performance_report.md` records a v2 frontend (MapLibre GL) that replaced *live GEE XYZ tiles* (inference per tile) with a *single pre-rendered overlay*. Measured on a 5.3 x 5.3 km AOI:

	v1 (36 live tiles)	v2 (1 overlay)
Earth Engine requests	36	1
Delivered	19 / 36 (53%)	1 / 1
Wall to done	~19.9 s	~19.7 s
Bytes	~78 KB (incomplete)	~38 KB (complete)

That report’s conclusion: wall-clock was a wash because both paths pay the same GEE inference; the change fixed completeness and flicker, not speed. The v2 code was later deleted from the working tree. The genuinely

new lever this roadmap adds is killing the per-pixel GEE inference itself and replacing “one big image” delivery with tiled delivery.

2. Diagnosis: where the 40 minutes and the memory go

Four independent bottlenecks:

1. **Per-tile re-inference on Earth Engine (the ~40 min).** Every 1792 px tile rebuilds the entire composite, indices, texture, SAR and classification server-side. GEE is metered by compute (EECU-hours); the interactive path is also capped by request lifetime (~5 min), response size, and concurrency. Client-side tricks cannot fix this.
2. **Low concurrency ceiling.** The project is on Earth Engine’s free Community Tier and has hit Restricted Mode: roughly 3 concurrent thumbnails with hard 429s. Adding workers/client threads does not help; the platform enforces the ceiling.
3. **A giant PNG as the delivery artifact.** ~100 MP, one payload, one `<img>`, ~420 MB decoded RGBA, re-rasterised on every pan/zoom. The memory blowout and most of the lag.
4. **No zoom-aware LOD.** The flat image forces the browser to push ~100 MP through the rasteriser even when the viewport shows a few hundred real pixels.

Any new architecture must satisfy the **10 m validity constraint** (never re-run or interpolate the classification itself) while fixing all four. The research threads below show the constraint is fully compatible with a COG/tile pipeline: classify once at 10 m, and everything downstream is label-preserving (NEAREST/MODE resampling, never interpolation).

3. Design goals and constraints

- **10 m/px only.** The classified raster is stored and served at its training scale. Coarser zooms are majority-class summaries, never re-inferences.
- **Map and numbers consistent.** The per-class area statistics must come from the same 10 m pixels the map shows; overviews are display-only.
- **Computed rarely, browsed a lot.** A district’s classification is paid for once, cached hash-keyed, then browsed at 60 fps.
- **Fit the existing stack.** Python + FastAPI, containerised, deployed on Render, static frontend.
- **Budget-conscious.** The current architecture burns GEE quota on every render; the target must not.

4. Recommended target architecture

The pipeline has five stages, each covered by a research thread below:

1. **Compute once.** One 10 m classification of the AOI (Section 5).
2. **Store as COG.** Cloud Optimized GeoTIFF: uint8 class ids, internal palette, MODE (majority) overviews (Section 6).
3. **Serve tiles.** TiTiler behind nginx + optional CDN (Section 7).

4. **Orchestrate as jobs.** Taskiq or Celery with Redis, a separate worker service (Section 8).
5. **Render in the browser.** MapLibre GL raster tile source (Section 9).

The data flow:

1. The user draws an AOI. The API hashes it with every pixel-affecting parameter into `cid` (the existing `overlay_cache_key`) and enqueues one job.
2. A worker classifies the AOI once at 10 m (GEE batch export, or full off-GEE inference) and publishes a COG.
3. The browser requests only the 256 px XYZ tiles it can see: `GET /maps/cid/z/x/y.png`, served by TiTiler from the COG, cached at nginx, CDN and browser.
4. The existing 5 s polling `/api/jobs/{cid}` drives the progress banner until the COG is published; after that tiles are available for everyone instantly.

Process topology: the API (uvicorn), TiTiler (own uvicorn process) and nginx share one “web” container/box; a second “worker” service (no public port) runs the long jobs; Redis holds queue + job state; the COG is authoritative in GCS with a local disk cache for the tile box. This keeps the heavy classifier out of the web process’s RAM.

5. Compute: killing the 40-minute wait

Three options, in increasing ambition. Options B and C compose on top of A.

5.1 Option A (stopgap, ~1-2 days): one Earth Engine batch export to COG

Today the AOI is cut into ~50 interactive `getThumbURL` requests, each re-running the chain and each subject to the 429 ceiling. The alternative: build the *same* classified image once and start a single `Export.image.toCloudStorage` task that writes a COG directly.

- GEE batch export runs server-side, auto-parallelises, and is **not** subject to the interactive concurrency ceiling; GEE documents ~2 concurrent batch tasks per project on average, up to 10-day lifetimes, auto-retry up to 5 times. The client-side 429/retry/backoff machinery disappears.
- `formatOptions: { cloudOptimized: true }` (or the newer COG file format) emits a Cloud Optimized GeoTIFF. Very large exports split into `name-yMin-xMin` chunk files that must be mosaicked into one COG with a VRT + `rio cogeo`.
- Monitor via EE task status; compute the class histogram locally with `numpy.bincount` on the COG instead of a synchronous `reduceRegion`.
- Cost and wall-clock: still comparable to today (~15-40 min) because the same pixels burn the same EE compute, and large exports bill EECU-hours. **Option A fixes fragility and produces the COG product; it does not kill the wait.**

5.2 Option B (recommended target, ~1.5-3 weeks): move feature building and inference off Earth Engine

GEE is kept only as the training-data source (one-time export of the labelled sample table); inference becomes local Python:

1. **Sentinel-2 composite from COGs.** The same data already exists as free COGs on Microsoft Planetary Computer (`sentinel-2-l2a` , uint16 x 0.0001 reflectance) and on AWS Element 84 Earth Search. `pystac-client` + `stackstac` builds a lazy dask/xarray stack; `.median(dim="time")` in windowed chunks gives the composite. A one-month Jabalpur window is ~8-14 granules, ~0.6-1.3 GB of network; ~2-4 min.
2. **SAR the same way.** `sentinel-1-grd` (VV/VH). Unit parity must be validated against the training table (dB vs amplitude conventions differ across providers) - flagged risk.
3. **19-band feature stack in numpy.** Port `_add_spectral_indices` / `_add_sar_bands` (pure arithmetic, exact parity). The only real work is GLCM texture: quantise B8/VV to 0-31 and compute the Haralick moments with shifted-array operations, ~1-2 min for 105 MP on 4-8 threads.
4. **Retrain locally.** GEE classifiers cannot be serialised/exported in a portable form (`.getInfo()` returns params, not trees), so export the training sample table once (`Export.table`) and retrain with scikit-learn `RandomForestClassifier` or XGBoost. 200k-1M rows x 19 features fits in seconds to ~2 min; XGBoost prediction over 105 M rows is ~30-90 s on CPUs, decided by memory bandwidth, not FLOPs.
5. **Infer to COG.** Windowed rasterio reads -> batching predict -> uint8 labels + `scipy.ndimage` focal smoothing -> `rio-cogeo` with MODE overviews. Histogram/areas from `numpy.bincount` , milliseconds.

Wall-clock target: ~4-6 min cold for a district (network-dominated), of which classification is ~30-90 s. GEE compute per render: ~0. This is the only option that structurally removes the 40-minute wait and the quota dependence. **Equivalent quality, not bit-identical output:** GEE's `smileRandomForest` (bagFraction 0.3) is not bit-for-bit reproducible in sklearn; validate with a class-proportion/Kappa gate on held-out districts before go-live.

5.3 Option C (multiplies B): cached per-district feature cubes

Cache the (district, month) 19-band feature stack: the first render in a month window pays fetch + features once; every later request (different model, adjusted dates within the cached month, rebuilds) is local compute. Coarse-resolution pyramids of the features must **never** feed the model - the 3x3-pixel GLCM and focal smoothing invalidate any resolution change (see the 5.7% vs 32% water-share note in the code). Caching at 10 m only.

5.4 Performance table (district-size AOI)

Stage	Today (tiled thumbnails)	A: batch COG export	B: local inference	B + C (feature cache)
Composite + features	re-run per tile (50x)	once, server-side	2-4 min (network)	0 (cached)
GLCM + indices	inside the ~2 min/tile	inside the task	1-2 min (numpy)	0
Classification	~2 min/tile	in task	30-90 s	30-90 s
COG assembly	PNG stitch	auto-COG (+ re-COG)	30-60 s	30-60 s
Wall clock	~40 min	~15-40 min	~4-6 min	~1.5-3 min
GEE quota per render	high	high	~0	~0
Requests to GEE	~50 + retries	1 + task poll	0	0

5.5 Compute references

- Earth Engine Exporting Images (COG output, splits, maxPixels): https://developers.google.com/earth-engine/guides/exporting_images
- Earth Engine Usage quota and limits: <https://developers.google.com/earth-engine/guides/usage>
- Earth Engine Noncommercial tiers and Restricted Mode: https://developers.google.com/earth-engine/guides/noncommercial_tiers
- Earth Engine computation benchmarks (EECU-hours for batch composite export): https://developers.google.com/earth-engine/guides/computation_benchmarks
- stackstac docs: <https://stackstac.readthedocs.io/>
- Planetary Computer S2/S1 collections: <https://planetarycomputer.microsoft.com/>
- rasterio windowed I/O: <https://rasterio.readthedocs.io/en/stable/topics/windowed-rw.html>

6. Storage: the Cloud Optimized GeoTIFF

The classified raster is a single-band **categorical** image: 5 class ids plus nodata for outside-the-AOI. COGs are the right container for three reasons: the browser/tiler reads only the blocks it needs via range requests, internal overviews give free low-zoom tiles, and the file is a single immutable object that caches perfectly.

Recommended encoding for this dataset:

- **One band, uint8** (values 0-4, nodata e.g. 250). Do not use 16-bit; it doubles decoded block sizes with no benefit.
- **Colormap**: TIFF palettes only store RGB (alpha is dropped by the GTiff writer), so encode RGBA at render time. Bake the class->RGBA legend into the tile style and serve alpha-transparent PNGs by combining the palette with the internal mask. Export the same palette to the frontend legend so map and statistics agree.
- **Compression**: ZSTD with horizontal `PREDICTOR=YES` - consecutive same-class runs compress to near-zero deltas, and ZSTD decodes fastest per tile (the tile path reads the COG on every first request). DEFLATE is the interoperability fallback; do not use JPEG/WEBP/LERC/JXL for a class raster (lossy or float-oriented).
- **Blocks**: 512 px main IFD, 128 px overview blocks (rio-cogeo defaults; matches COG spec guidance).
- **Overviews**: factors 2 down to 128 with **MODE (majority) resampling** (`--overview-resampling mode`). Every overview pixel is a real class id - at low zoom you see the dominant class, never a blend. NEAREST is also “valid” but speckles small polygons at coarse zooms. GDAL defaults warps of paletted images to NEAREST; be explicit anyway. **Overviews are display-only**: areas/histograms must come from the full-resolution level (or the GEE histogram as today).
- **Mask**: add an internal 1-bit mask (`--add-mask`) so everything outside the AOI polygon is transparent, keyed on the same outline the model predicts.
- **Web-Mercator alignment**: build the COG on the Google Maps compatible grid (`--tms WebMercatorQuad --web-optimized`) so each XYZ tile request maps to 1-4 internal blocks and the per-tile warp cost is near zero. Border pixels can shift under a pixel versus the native grid; keep the raw 10 m export as the analysis copy if needed.

The one-off build command (after a GEE export, or after local inference):

```
rio cogeo create classified.vrt classified_cog.tif \
  --cog-profile zstd --blocksize 512 --overview-resampling mode \
  --overview-blocksize 128 --nodata 250 --add-mask \
  --co PREDICTOR=YES --tms WebMercatorQuad --web-optimized
rio cogeo validate classified_cog.tif
```

Expected size for a district: ~105 MP single band uncompressed is ~100 MB; with ZSTD on long same-class runs, roughly **5-30 MB** plus 5-15% for overviews (verify once with `du`).

6.1 Storage COG references

- rio-cogeo: <https://cogeotiff.github.io/rio-cogeo/>
- GDAL COG driver: <https://gdal.org/en/stable/drivers/raster/cog.html>
- COG spec (OGC 21-026): <https://docs.ogc.org/is/21-026/21-026.html>
- cogeo.org primer: <https://www.cogeo.org/>

7. Serving: TiTiler behind nginx + CDN

7.1 Why TiTiler

TiTiler (Development Seed) is a FastAPI service that serves 256 px XYZ/WebMercator tiles directly from a COG on disk or via `/vsicurl` from object storage. It is Python/FastAPI (same stack as this app), automatically uses the COG's internal overviews, and auto-loads the internal colormap. It also serves `tilejson.json`, metadata and statistics endpoints. Measured by the project: ~20-30 ms average per tile, ~54 KB per tile, on ordinary runners (its own published benchmark page).

Ranking against alternatives for this categorical, one-artifact-per-AOI use case:

Option	Verdict
TiTiler on-the-fly	Recommended. The COG overview pyramid <i>is</i> the pre-baked pyramid; first request encodes a tile, every repeat is absorbed by caches.
Static pre-baked tile bucket (gdal2tiles/TiTiler loop)	Fine if traffic explodes to thousands of req/s; otherwise redundant bytes with no benefit - the COG already holds the pyramid.
MapProxy	Its value is seeding/cache management; here it would duplicate nginx/CDN caching for no benefit.
GeoServer/MapServer	Java/C stack for one dataset; no native XYZ; overkill.
Terracotta	Viable but adds a SQLite tile index and its own conventions for no gain here.

7.2 Process shape and GDAL tuning

Run TiTiler as its **own uvicorn process** (not mounted into the API app): GDAL config is process-global, the worker-style classifier load should not share RAM with the tile path, and nginx `proxy_cache` then sits cleanly in front of just `/maps/*`.

- `uvicorn titiler_app:app --workers 1` on a ~1 GB box. Rasterio delegates decode to C and releases the GIL, so one worker with uvicorn’s threadpool serves 25-60 tiles/s from warm disk - far more than this scale needs.
- Install `titiler.core` (the `titiler` metapackage was dropped in late 2025). Optional in-memory tile cache via the documented “Tiler with Cache” aiocache pattern.
- GDAL env for local-disk COGs: `GDAL_CACHEMAX=200`, `VSI_CACHE=TRUE`, `VSI_CACHE_SIZE=5000000`, `GDAL_DISABLE_READDIR_ON_OPEN=EMPTY_DIR`, `GDAL_SKIP=VRT`, `GDAL_PAM_ENABLED=NO`, `GDAL_HTTP_RETRY` vars if reading via vsicurl later.
- **SSRF hardening**: never expose TiTiler’s arbitrary `url=` parameter. Frontend asks only for `/maps/{cid}/z/x/y.png`; a custom path dependency resolves `cid` to a whitelisted file under the COG directory (or exact GCS object). Keep `GDAL_SKIP=VRT` and CORS tight.
- Serve the COG already RGBA-colormapped so tile URLs have **zero query parameters** - canonical URLs are what make every cache layer effective.

7.3 Cache hierarchy

Layer	Setting
TiTiler response	<code>Cache-Control: public, max-age=31536000, s-maxage=31536000, immutable, no-transform</code> (no-transform stops CDN image recompression).
CDN (Cloudflare)	Origin Cache-Control passes through (free tier); tiles cached at edge.
nginx	<code>proxy_cache</code> disk zone (e.g. 2 GB) keyed on the full URI for <code>/maps/*</code> ; survives restarts; adds <code>X-Cache-Status</code> .
Browser	MapLibre in-memory tile store + <code>immutable</code> avoids revalidation on pan-back.

Because `cid` is a hash of every pixel-affecting parameter, bytes at a URL never change, so `immutable` is truthful. A Redis tile cache adds nothing beyond nginx here - skip it unless multiple TiTiler replicas share no proxy.

7.4 Serving references

- TiTiler: <https://developmentseed.org/titiler/>
- TiTiler performance tuning + GDAL config: https://developmentseed.org/titiler/advanced/performance_tuning/
- TiTiler security (host allowlist, VRT): <https://developmentseed.org/titiler/security/>
- nginx proxy module: https://nginx.org/en/docs/http/nginx_http_proxy_module.html
- Cloudflare cache-control behavior: <https://developers.cloudflare.com/cache/concepts/cache-control/>

8. Orchestration: long jobs that survive restarts

Jobs are few (user-triggered), last 10-40 min, and are I/O-bound on the remote EE API. The current in-process thread + dict fails on any restart and shares RAM with the web process.

Recommendation: Taskiq (asyncio-native) or Celery with a Redis broker/result backend, run as a separate Render `worker` service with no public port; API stays thin.

- **Why not the alternatives.** FastAPI `BackgroundTasks` and plain asyncio tasks are in-process and lost on restart. RQ and Dramatiq are viable but sync-oriented. ARQ has ideal semantics (job-id dedupe, keep_result, deferred retry) but the maintainer has put it in maintenance-only mode. Taskiq keeps those semantics with a small asyncio footprint; Celery is the “boring, everyone-knows-it” fallback. Either is correct here.
- **Idempotency is the design.** Every Redis queue re-runs a job if the worker dies mid-job (“jobs may be called more than once” - stated in ARQ docs and true of the pattern). Make `job_id = cid` (the content hash) so a re-run recomputes byte-identical artifacts; the job starts by checking “COG already published?” and returns early if so. Publish the COG atomically, write the `done` marker last.
- **Status model** (keeps the frontend poll contract): queued, exporting_on_ee (EE task poll), building_cog, publishing, done / failed. Expose via `/api/jobs/{cid}` reading Redis + a COG existence check. Keep the existing 5 s polling; SSE/WebSocket is an optional later upgrade that buys nothing at single-digit concurrent users.
- **Redis, not Postgres/filesystem JSON**, for state: filesystem JSON dies with the instance; Postgres buys nothing for short-lived key/value state. Render offers managed Key Value; a native single box could also use its own small Redis.
- **Restart handling:** queued jobs survive by construction; a startup hook on the worker re-enqueues anything claimed-but-not-done that has no publish marker, so every job resolves to done or failed, never silent limbo.
- **Renderer notes for Render specifically:** free-tier web instances spin down and free workers cannot sustain 10-40 min jobs - the worker must run on a paid instance. A Render persistent disk blocks zero-downtime deploys and horizontal scaling; prefer GCS for the authoritative COG and use ephemeral disk only as a tile-side cache (a miss costs one range request to GCS).

Job-state transition to keep: REQUEST -> hash to cid -> enqueue -> poll `/api/jobs/cid` until done -> map points at `/maps/cid/...` The frontend already polls; the change is additive.

8.1 Orchestration references

- Taskiq: <https://github.com/taskiq/taskiq>
- Celery: <https://docs.celeryq.dev/en/stable/>
- ARQ (maintenance notice issue #510): <https://github.com/python-arq/arq>
- Render background workers: <https://docs.render.com/background-workers>
- Render disks (ephemeral FS, single instance): <https://docs.render.com/disks>

9. Client: MapLibre GL raster tiles

9.1 Recommended stack

MapLibre GL JS with a `raster` source pointing at the TiTiler/XYZ tile pyramid. Browsers decode the PNG tiles off the main thread and MapLibre uploads them straight to GPU textures from a pooled, LRU-cached set, with per-frame network scheduling (16 parallel, 8/frame). No JS decode, one WebGL context, bounded memory. Do not decode the COG in the browser as the base path: keep that as a fallback only.

Low-zoom tiles come from the COG's MODE overviews, native zoom comes from the 10 m level, and the palette is baked into the tiles, so the browser never re-classifies and never runs the model.

9.2 Source and layer settings

- Source: `tiles: [url/maps/{cid}/{z}/{x}/{y}.png]`, `tileSize: 256` (MapLibre raster defaults to 512; 256 prevents upscaling blur), `bounds: [sw.lng, sw.lat, ne.lng, ne.lat]` so no tile outside the AOI is requested, `minzoom: 0`, `maxzoom: 14` (native LOD for 10 m at this latitude; everything below comes from overviews).
- `paint["raster-resampling"] = "nearest"` - the single most important correctness setting. `linear` (default) blends class colours across borders (Forest into Water) and shows classes the model never predicted.
- `paint["raster-fade-duration"] = 0` to avoid two LODs ghosting through each other mid-zoom.
- `paint["raster-opacity"]` drives the existing transparency slider. Keep `raster-brightness/saturation/contrast` at 0.
- Cap map zoom at ~z14 or accept overzoomed z14 tiles above it; decide per product.
- Basemap below, classified layer above, no layer-order fights.

9.3 Memory budget

- A 256 px RGBA tile is ~256 KB constant, ~341 KB with mipmaps. A 1080p viewport showing ~9-16 tiles plus an eviction ring of ~40-60 cached tiles is **~15-25 MB of GPU texture memory total**, independent of the 105 MP dataset, because only visible tiles materialise (versus ~420 MB today).
- Map constructor `maxTileCacheSize` defaults to a viewport-based dynamic value (ZoomLevels 5); measured fine here. Hard-cap only if DevTools shows unbounded growth.
- Premultiplied alpha defaults are correct for compositing the RGBA palette; leave them.
- Serve tiles `immutable` +ETag so the browser HTTP cache never revalidates on pan-back.

9.4 Alternative clients ranked

Option	Verdict
MapLibre GL + raster tiles	Recommended; GPU textures, built-in cache/scheduling, nearest resampling.
MapLibre + COG-in-browser (maplibre-cog-protocol, geotiff.js range reads)	Fallback if tiles are impossible; adds per-tile JS decode cost and requires EPSG:3857 COG + CORS + range support on the object store.

Option	Verdict
OpenLayers GeoTIFF source + WebGLTileLayer	Good native COG decimation, but more moving parts for a categorical raster.
Leaflet + georaster-layer-for-leaflet / TileLayer.GL	CPU-bound decode on pan; experimental WebGL plugins lack their own cache.
deck.gl	Geo-tile reimplementation required; not worth it for a 2D overlay.

9.5 Client references

- MapLibre style spec (raster sources/layers): <https://maplibre.org/maplibre-style-spec/sources/#raster>
- MapLibre GL JS source (tile cache, request scheduling): <https://github.com/maplibre/maplibre-gl-js>
- maplibre-cog-protocol (COG fallback): <https://github.com/geomatico/maplibre-cog-protocol>
- geotiff.js (browser range reads): <https://github.com/geotiffjs/geotiff.js>
- OpenLayers COG example: <https://openlayers.org/en/latest/examples/cog.html>

10. Consolidated expected performance

All figures below are research estimates; the ones marked “measure” must be confirmed once built.

Metric	Today	Target (off-GEE + COG + MapLibre)
District end-to-end	~40 min	~2-6 min cold, ~1.5-3 min cached feature cube
Repeat request of same AOI	re-computed	instant, hash-keyed
Time to first tiles after job done	waits for full PNG	seconds; tiles stream in
Cold unique tile latency	whole-image	~20-70 ms; measured by TiTiler
Browser GPU memory for overlay	~420 MB decoded	~15-25 MB textures
Pan/zoom	full-image re-raster	60 fps, visible tiles only
Earth Engine quota per render	high (Restricted Mode every time)	~0 on off-GEE path
Class statistics	~1-2 s reduceRegion + retries	milliseconds (numpy.bincount)
Storage per AOI	giant PNG	5-30 MB COG (measure)
Operability	one resurrectable daemon thread, restart loses jobs	durable jobs in Redis, separate worker

Worked sizing for Jabalpur: full-res ~13,700 x 7,700 px (~105 MP). Mercator pyramid has a few hundred z14 tiles across the AOI; a user viewport draws 20-80 tiles per screenful. One small box + TiTiler serves dozens of concurrent users; tile throughput is ~25-60 tiles/s worker cold, cached after first visit.

Clip notes on honesty: step A alone measurably cuts fragility but not wall-clock; quality parity between GEE sklearn retraining is “statistically equivalent”, not byte-identical - validated with a gate. EECU-hour figures for a

district-scale render are estimates derived from GEE's published batch-composite benchmarks, not measured for this exact workload.

11. Migration roadmap

Recommended sequencing, each phase independently shippable:

Phase 1 - COG + tile delivery (1-2 weeks, biggest immediate win). - Replace the PNG stitch with the tiled rendering: Option A export task -> COGify script -> TiTiler service + nginx + cache headers. - Swap the frontend from `imageOverlay` to a MapLibre `raster` source over `/maps/{cid}/...` - Keep `/api/classify` + polling contract so analytics and flow are unchanged; the map simply gets tiles instead of a giant PNG. - Delete `render_overlay_png` / `_fetch_tile` / `/overlays` from the request path. - Result: map is 60 fps and light, first-paint is seconds, the 40 min render becomes a transparent async wait, and repeats are instant.

Phase 2 - durable jobs (2-4 days). - Add Redis + Taskiq/Celery worker service on Render (paid instance), move the classify+export into the worker, `/api/classify` becomes enqueue + idempotent dedupe by cid.

Phase 3 - kill the compute wait (1.5-3 weeks). - Export the training table once, retrain RandomForest/XGBoost in Python, port `_add_spectral_indices` / `_add_sar_bands` to numpy, infer via STAC/stackstac. Validate with the class-proportion/Kappa gate against GEE output on held-out districts. - Optionally cache per-district feature cubes (Option C) for ~2-3x further latency reduction.

Phase 4 - polish and observe. - prometheus-fastapi-instrumentator on API + TiTiler, structlog with cid/stage, an nginx tile-latency log, and a per-cid tile-coverage sweep so map holes surface as metrics, not screenshots.

12. Risks and open questions

1. **Model parity (highest risk).** GEE `smileRandomForest` is not bit-for-bit reproducible; the validation gate is a sign-off, not a checkbox. If identical output is a hard requirement, stop at Phase 2 (Option A keeps GEE inference).
2. **SAR unit parity off-GEE.** VV/VH amplitude/gamma0 conventions differ across catalogs; verify against the exported training table before trusting the 6 SAR features.
3. **Cloud-masking parity.** GEE QA60 vs Planetary Computer SCL mask differently; validate on training samples.
4. **COG overview resampling.** MODE only; a bilinear/mean overview silently produces invalid class ids at low zoom. Validate every published COG (Rio cogeio validator) before flipping `done`.
5. **GEE export chunking.** Very large exports split into multiple files; mosaic via VRT before COGify. Confirm 105 MP fits a single task's maxPixels.
6. **GEE quotas persist.** Batch tasks still burn EECU-hours and collect on the same meter; Restricted Mode can slow batch tasks without killing them. Option A is a fragility fix, not a quota fix.
7. **Render persistence.** Free Render instances spin down and cannot run 10-40 min jobs; the worker needs a paid instance. GCS for authority, ephemeral disk for the tile cache.
8. **SSRF.** Never reintroduce TiTiler's raw `url=` endpoint; resolve `cid` server-side to a whitelist.
9. **Open questions to close before building:** size and class balance of the training table; whether "identical output" or "statistically equivalent" is acceptable; which S2/S1 catalog is canonical; egress budget of the worker VM (~1-2 GB per district render).

13. Technology stack summary and research basis

Decisions made by this research, with the alternatives considered:

Concern	Chosen	Considered and rejected
Raster container	Cloud Optimized GeoTIFF (uint8, palette, ZSTD, MODE overviews, mask)	giant PNG, flat GeoTIFF
Tile serving	TiTiler on-the-fly	MapProxy, GeoServer/MapServer, Terracotta, static tile bucket, MapTiler cloud
Browser renderer	MapLibre GL raster source	OpenLayers, Leaflet+plugins, deck.gl, geotiff.js/direct-COG
Compute	Phase 1: GEE batch export; Phase 3: local inference (STAC + numpy + sklearn/XGB)	per-tile thumbnails (today), high-volume computePixels, GPU/ONNX tree inference
Orchestration	Taskiq (or Celery) + Redis	FastAPI BackgroundTasks, asyncio threads, ARQ (maintenance mode), RQ/Dramatiq
Cache layers	browser immutable -> Cloudflare -> nginx proxy_cache	Redis tile cache (unneeded at this scale)
Job/COG persistence	Redis state + GCS authoritative COG, local disk cache	filesystem JSON, Render persistent disk
Observability	prometheus-fastapi-instrumentator + structlog + nginx logs	full APM

Reliability of this report's claims: statements about COG structure, GDAL/rio-cogeo behaviour, TiTiler endpoints/config, GDAL env vars, MapLibre cache/resampling semantics, Redis-queue re-run semantics, and Render deployment constraints were checked against primary documentation (links in each section). Performance figures for tile latency, GLCM runtime, XGBoost inference, and EECU-hour burn are research estimates from published benchmarks and should be treated as targets to measure, not guarantees.

A short operationally-focused appendix note on why the whole thing can be built in Phases: the single most important decision is Phase 1 (COG + tiles + MapLibre), because it is what fixes the user-visible lag and memory today and it requires zero model change. Everything after that reduces the wait before the map appears.